Chapitre 2:
Les conteneurs

Plan



1. Les conteneurs
2. Les conteneurs séquentiels
    2.1. Les chaines de caractères
    2.2. Les listes
    2.3. Les tuples
3. Les conteneurs à accès direct
    3.1. Les ensembles
    3.2. Les dictionnaires

Les conteneurs

Les conteneurs

Python permet de regrouper **un ensemble de données de même type ou de types différents** à l'intérieur de structures de données spéciales appelées **conteneurs**. Il existe deux catégories de conteneurs :

- **Les conteneurs séquentiels :**
  - Une séquence est un conteneur **ordonné** d'éléments **indexés par des entiers** indiquant leur position dans le conteneur (commençant de 0)
  - 3 types prédéfinis de séquences : les chaines de caractères, les listes et les tuples.
- **Les conteneurs à accès directs :** Dictionnaires et ensembles

Les conteneurs séquentiels

Le type : chaîne de caractère

- Une chaîne de caractères est une séquence ordonnée de caractères **immutables (non modifiables).**
- On ne peut donc pas modifier un ou plusieurs caractères. Si on ajoute un caractère (ou une autre chaîne) par concaténation à une chaîne existante, le résultat est la création d'une nouvelle chaîne à une nouvelle adresse en mémoire.

- Une chaîne de caractères peut être définie de différentes manières :
  - Entre guillemets
  - Entre apostrophes (guillemets simples)
  - Entre triple guillemets

- Le choix est souvent imposé par le contenu de la chaîne : une chaîne contenant des guillemets simples sera déclarée avec des guillemets doubles et réciproquement.

Concaténation des chaînes de caractères

- La concaténation est l'opération qui consiste à construire une chaîne de caractères en plaçant cote à cote deux autres chaînes de caractères (opérateur **+**).

```
In [27]: chaine1='bonjour'

In [28]: chaine2='TIC A'

In [29]: maChaine=chaine1 + ' ' +chaine2

In [30]: print(maChaine)
bonjour TIC A
```

Répétition des chaînes de caractères

- Une chaîne de caractères peut être créée par concaténations multiples de la valeur d'une autre chaîne. Le nombre de concaténations étant spécifié par une valeur entière (opérateur * avant ou après la chaine).

```
In [38]: chaine1='bonjour'

In [39]: chaine2='TIC A'

In [40]: ch=chaine1 + ' ' + chaine2

In [41]: print(ch*3)
bonjour TIC Abonjour TIC Abonjour TIC A
```

Manipulation des chaînes de caractères

- Python offre une méthode simple pour accéder aux caractères contenus dans une chaîne : une chaîne est manipulée comme une séquence indexée de caractères. Ainsi, chaque caractère est accessible directement par son index (**le premier étant indexé 0)** en utilisation des crochets.

- En plus de cet accès unitaire aux caractères, il est possible d'accéder à des sous-chaînes en précisant la tranche souhaitée l'index de début (qui est inclus) étant séparé de l'index de fin (**qui est exclu**) par le caractère **:**

```
In [44]: x='tic A TEKUP'

In [45]: len(x)
Out[45]: 11

In [46]: x[4]
Out[46]: 'A'

In [47]: x[0]
Out[47]: 't'

In [48]: x[0:3]
Out[48]: 'tic'
```

Manipulation des chaînes de caractères

- Un index négatif indique que le calcul s'effectue à partir de la fin de la chaine :

```
In [50]: x
Out[50]: 'tic A TEKUP'

In [51]: x[-1]
Out[51]: 'P'

In [52]: x[-3]
Out[52]: 'K'

In [53]: x[1:-1]
Out[53]: 'ic A TEKU'
```

Méthodes prédéfinies sur les chaînes de caractères

- `'N' in s` : renvoie True si N est un caractère de s.
- `s.count('ab')` : retourne le nombre de chaîne "ab" (non overlappant).
- isalnum, isalpha, isdigit, islower, isupper, isspace : tests sur le type des caractères de la chaîne (False si chaîne vide).
- `s.startswith('ab')` : renvoie True si commence par "ab" (idem avec endswith).
- `s.find('ab')` : retourne le plus petit index correspondant à la chaîne "ab" (-1 si pas trouvé).
- `s.find('ab', 3)` : retourne le plus petit index correspondant à la chaîne "ab" dans s[3:] (-1 si pas trouvé).
- `s.find('ab', 3, 15)` : retourne le plus petit index correspondant à la chaîne "ab" dans s[3:15] (-1 si pas trouvé).
- `s.rfind('ab')` : retourne le plus grand index correspondant à la chaîne "ab" (-1 si pas trouvé), i.e commence par la recherche par la fin
- `s.index('ab')` comme find, mais si sous-chaîne non trouvé, lève une ValueError.
- `s.lower()` : renvoie la chaîne convertie en minuscules (idem avec `s.upper()` pour les majuscules), sans affecter s
- `s.capitalize()` : met en majuscule la première lettre et en minuscules toutes les autres quelque soit la casse de départ.
- `s.title()` : renvoie une chaîne où toutes les premières lettres de chaque mot sont en majuscule et le reste en minuscules.

Méthodes prédéfinies sur les chaînes de caractères

- `s.replace('old', 'new')` : remplace toutes les occurrences de "old" par "new", sans affecter s.
- `s.replace('old', 'new', 1)` : remplace seulement la première occurrence.
- `s.translate(str.maketrans('ACGT', 'TGCA'))` : renvoie la chaîne avec les A remplacés par des T, les C par des G, etc (maketrans construit une table de traduction des 255 caractères)
- `s.translate(None, '\n')` : enlève les retours chariots, sans affecter s.
- `s.translate(None, 'aeiouy')` : enlève les voyelles.
- `str.split(s)` : retourne une liste de chaînes en coupant sur les caractères blancs (espace, tabulation, retours chariot). Si s est vide, la liste est vide.
- on peut aussi faire `s.split()`. Par exemple, `' a b c '.split()` donne `['a', 'b', 'c']`. Donc attention, split se comporte différemment sans argument ou avec une chaîne ayant un espace :
  - `' a b'.split(' ')` : renvoie `['', 'a', '', 'b']`
  - `' a b'.split()` : renvoie `['a', 'b']` : les chaines vides sont éliminés !
- `s.split('\t')` : splitte sur les tabulations, et se comporte bien par défaut si à la fin de la chaîne, il y a des tabulations consécutives (contrairement à perl). Attention, c'est une chaîne qui est utilisée et non un pattern pour indiquer où couper.

Méthodes prédéfinies sur les chaînes de caractères

- `str.split(s, 'a')` : splitte sur les "a" (chaîne). Si s est vide, la liste comporte un élément, la chaîne vide.
- `sep.join(stringList)` : concatène les chaînes stringList avec la chaîne sep comme séparateur.
- `s.strip()` : retourne un chaîne avec les espaces à gauche et à droite retirés (trim), y compris les retours chariot (sinon, `rstrip` ou `lstrip` pour un seul côté).
- `s.strip('a')` : retourne une chaîne avec les 'a' des deux côtés enlevés (pareil avec lstrip et rstrip)
- `s.center(15)` : retourne une chaîne de longueur 15 avec des espaces pour centrer la chaîne (si s est de longueur > 15, retourn s). `s.center(15, '-')` : utilise '-' au lieu d'un espace.
- `s.ljust(15)` : justification à gauche (rjust pour à droite), sans affecter s
- `ord('A')` : renvoie 65 (conversion d'un caractère en code décimal).
- `chr(65)` : renvoie 'A' (conversion d'un code décimal en caractère, fonction inverse de la fonction ord).

Autres opérations pour le type str

Opération	Résultat
s.endswith(suffix)	True si la chaîne s se termine par suffix
s.startswith(prefix)	True si la chaîne s commence par prefix
s.isalpha()	True si s est formée uniquement de caractères alphabétiques
s.isdigit()	True si s est formée uniquement de chiffres
s.isalnum()	True si s est formée uniquement de caractères alphanumériques
s.lower()	Une chaîne obtenue par passage en minuscules
s.upper()	Une chaîne obtenue par passage en majuscules
s.strip()	Une chaîne obtenue en supprimant les blancs au début et à la fin
s.replace(old,new,n)	Une chaîne obtenue en remplaçant old par new, au plus n fois
s.find(sub)	Indice de la première occurence de sub (et -1 si non trouvé)
s.index(sub)	Comme find, mais erreur ValueError si non trouvé
s.rfind(sub)	Indice de la dernière occurence de sub (et -1 si non trouvé)